PPO and SAC: A Comparative Study

Sergio Herreros Pérez [1] **Adrián López-Lanchares Echezarreta** [1] **Ignacio Bayón Jiménez-Ugarte** [1]

Abstract

We study the application of Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) to pixel-based continuous control tasks. While theoretically superior for continuous action spaces, PPO and SAC exhibit extreme hyperparameter instability when learning directly from high-dimensional visual observations. To address this, we leverage the Optuna framework to execute automated, short-horizon hyperparameter searches, rapidly identifying and pruning diverging configurations. We successfully scale these algorithms to full training horizons and compare their performance, sample efficiency, and stability against our prior Q-learning baselines.

1. Introduction

In our previous report, we demonstrated that value-based methods, such as Rainbow DQN, can successfully solve pixel-based continuous control tasks. However, adapting Q-learning to continuous domains requires action space discretization and even complex branching architectures to scale to more complex tasks. A more mathematically natural approach lies in policy gradient and actor-critic methodologies, which directly output continuous action distributions and are the standard choice for robotic locomotion.

In this study, we evaluate two state-of-the-art continuous control algorithms: Proximal Policy Optimization (PPO) (Schulman et al., 2017) and Soft Actor-Critic (SAC) (**?**). While these algorithms represent the theoretical gold standard for physical control tasks, applying them directly to high-dimensional visual observations introduces deep practical challenges. Specifically, we found both algorithms to be highly sensitive to their underlying hyperparameter configurations. Even on the relatively simple Hopper environment, establishing a stable learning signal proved remarkably difficult. For instance, PPO exhibits extreme sensitivity to its target Kullback-Leibler (KL) divergence penalty; a threshold set too high causes the value loss to explosively diverge, while a threshold set too low results in premature policy collapse and a complete failure to learn.

To navigate this extreme hyperparameter sensitivity, we pivot our methodology from manual heuristic tuning to automated, large-scale search using the Optuna optimization framework (**?**). Given the computationally expensive nature of pixel-based rendering, committing to long training horizons for unverified configurations is intractable. Instead, we designed an optimization pipeline that executes dozens of short-horizon trials. By evaluating the early learning dynamics over fewer environment steps, Optuna allows us to rapidly prune configurations that catastrophically diverge or stagnate. This broad but shallow search strategy effectively isolates the narrow, stable hyperparameter regimes required before scaling up to full training horizons.

2. Proximal Policy Optimization (PPO)

2.1. Algorithm Overview

Proximal Policy Optimization (PPO) (Schulman et al., 2017) is an on-policy policy gradient method that directly optimizes a stochastic policy $\pi_\theta(a|s)$ by maximizing the expected discounted return. To prevent destructively large policy updates, PPO introduces a clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \right) \right], \tag{1}$$

where $r_t(\theta) = {}^{\pi_\theta(a_t|s_t)}/_{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between the current and old policy, $\hat{A}_t$ is an estimate of the advantage function, and ε is the clipping threshold. This formulation effectively constrains policy updates to a trust region without requiring second-order optimization.

PPO jointly optimizes a combined loss that also includes a value function term and an entropy bonus to encourage exploration:

$$\mathcal{L}(\theta) = \mathcal{L}^{\text{CLIP}}(\theta) - c_1\mathcal{L}^{\text{VF}}(\theta) + c_2\mathcal{H}[\pi_\theta], \tag{2}$$

where c_1 and c_2 are coefficients controlling the relative contribution of the value loss and the entropy bonus respectively.

[1]Master in Artificial Intelligence, Universidad Pontificia Comillas (ICAI). Correspondence to: Sergio Herreros Pérez <sergio.herreros@alu.comillas.edu>, Adrián López-Lanchares Echezarreta <adrianlanchares@alu.icai.comillas.edu>, Ignacio Bayón Jiménez-Ugarte <ibayon@alu.comillas.edu>.

Advantages are estimated using Generalized Advantage Estimation (GAE) (Schulman et al., 2018), which balances bias and variance through a parameter λ:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \quad (3)$$

Unlike DQN, PPO operates natively in continuous action spaces by parameterizing the policy as a Gaussian distribution over actions, making it directly applicable to continuous control tasks without any discretization.

2.2. Hopper: Learning from Pixels

We validate PPO on the same Hopper task described in the previous lab excercise, maintaining identical environment and reward configurations to enable a fair comparison with BDQN.

Environment Configuration: The agent observes 64×64 pixel inputs with a frame stack of 4. We deploy 30 parallel environments to increase data throughput and stabilize advantage estimation. The `healthy_angle_range` is widened to $[-0.4, 0.4]$ to encourage diverse state exploration early in training.

Reward Configuration: The reward shaping follows the same formulation as before, targeting a forward velocity of 1.5 units/s with $w_{\text{vel}} = 1.0$, a control smoothness penalty of $w_{\text{ctrl}} = 0.001$, and a healthy survival bonus of $w_{\text{healthy}} = 1.0$.

Algorithm Configuration: PPO is trained for 1M environment frames. Each rollout collects 256 steps per parallel environment before performing 10 epochs of minibatch updates. The clipping threshold is set to $\varepsilon = 0.1$, the GAE parameter to $\lambda = 0.975$, and the discount factor to $\gamma = 0.995$. The policy and value networks share a convolutional encoder, with separate MLP heads for the policy mean and value estimate. Optimization uses Adam with a learning rate of 4×10^{-4}.

[*TODO: Describe convergence behavior here: does the agent reach the target velocity? How does sample efficiency compare to BDQN? Note any instabilities or failure modes observed during training.*]

2.3. Humanoid: Scaling to High-Dimensional Control

Following the Hopper experiments, we scale PPO to the Humanoid environment. Unlike BDQN, PPO avoids the combinatorial action-space explosion by directly outputting a continuous Gaussian policy over all joint torques, making it inherently better suited to high-dimensional continuous control.

Environment Tweaks: We apply the same morphological simplifications as in the previous lab excercise: the Z-axis rotations of the abdomen and hips are frozen, all six arm joints are disabled, and the observation resolution is increased to 96×96 pixels to capture the finer postural nuances of the humanoid body.

Algorithm Configuration: [*TODO: Describe the specific hyperparameter configuration used for Humanoid: number of parallel envs, rollout length, number of update epochs, learning rate, clipping threshold, entropy coefficient, and any curriculum or reward shaping adjustments relative to Hopper.*]

[*TODO: Describe results here: did PPO successfully learn to walk? How many frames were required? Compare qualitatively and quantitatively to the BDQN Humanoid failure. Discuss whether the continuous action space parameterization provided a meaningful advantage.*]

3. Soft Actor-Critic

3.1. Methodology

We implement Soft Actor-Critic (SAC) (Haarnoja et al., 2018) for continuous control from image-based observations. SAC is an off-policy actor-critic algorithm that optimizes a stochastic policy under the maximum entropy reinforcement learning framework, encouraging both high reward and exploration.

Architecture. Our model consists of a stochastic actor and a double Q-function critic. Observations are high-dimensional images and are first processed through a convolutional encoder that extracts latent features.

The actor is parameterized as a Gaussian policy. The encoded features are passed through a multi-layer perceptron (MLP) to produce the mean and log-standard deviation of the action distribution. Actions are sampled using the reparameterization trick and passed through a $\tanh$ squashing function to ensure they lie within valid bounds. A correction term is applied to the log-probability to account for this transformation.

The critic is implemented as a **double Q-network** to reduce overestimation bias. A shared encoder extracts latent features, which are concatenated with the action and passed through two independent Q-heads, producing Q_1 and Q_2. During training, the minimum of these estimates is used to compute targets.

Replay Buffer. We use a uniform replay buffer with fixed capacity to store transitions (s, a, r, s', d). The buffer is implemented using preallocated tensors and a circular indexing scheme for efficiency. Observations are stored as `uint8` images and normalized to $[0, 1]$ at training time to reduce memory usage.

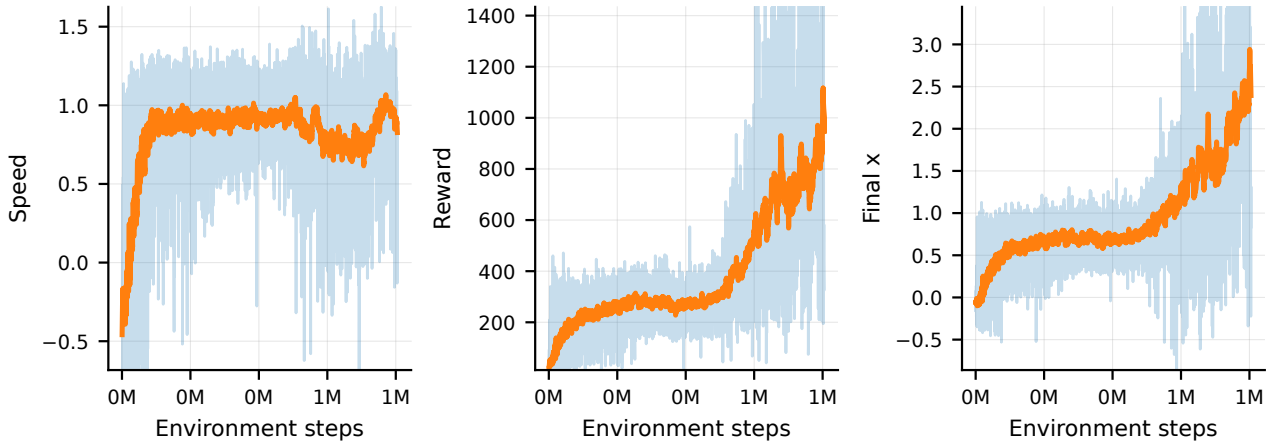

Figure 1. Training dynamics of PPO on the Hopper environment. (smoothed with an exponential moving average).

[TODO: Insert plot: episodic reward and/or gait metrics over training steps for PPO on Humanoid.]

Figure 2. Training dynamics of PPO on the Humanoid environment.

Training Objective. SAC optimizes a soft Bellman objective. The critic is trained by minimizing:

$$\mathcal{L}_Q = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[(Q_i(s,a) - y)^2\right], \tag{4}$$

where the target is:

$$y = r + \gamma(1-d)\left(\min_j Q_j(s',a') - \alpha \log \pi(a'|s')\right). \tag{5}$$

The actor is trained to maximize expected Q-values while encouraging entropy:

$$\mathcal{L}_\pi = \mathbb{E}_{s\sim\mathcal{D}}\left[\alpha \log \pi(a|s) - \min(Q_1, Q_2)(s,a)\right]. \tag{6}$$

Entropy Temperature. We support both fixed and automatically learned entropy temperature α. When enabled, α is optimized via:

$$\mathcal{L}_\alpha = -\mathbb{E}_{a\sim\pi}\left[\alpha(\log \pi(a|s) + \mathcal{H}_{\text{target}})\right], \tag{7}$$

where $\mathcal{H}_{\text{target}}$ is typically set proportional to the action dimension, allowing adaptive control of exploration.

Training Procedure. Training follows an off-policy loop with interleaved data collection and updates. Multiple environments are run in parallel to improve sample efficiency. Initially, actions are sampled randomly to populate the replay buffer; afterwards, actions are drawn from the learned policy.

At each iteration, a batch is sampled from the replay buffer, the critic is updated using the soft Bellman target, and the actor is updated using the entropy-regularized objective. Target networks are updated using Polyak averaging:

$$\theta_{\text{target}} \leftarrow \tau\theta + (1-\tau)\theta_{\text{target}}. \tag{8}$$

Gradient clipping is optionally applied for stability, and the number of gradient updates per environment step is configurable.

Implementation Details. Observations are normalized to $[0, 1]$ during training. The method assumes continuous action spaces, as required by SAC. Training progress is monitored via TensorBoard, and models are periodically checkpointed for reproducibility.

References

Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. 2018. URL https://arxiv.org/abs/1801.01290.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. 2017. URL https://arxiv.org/abs/1707.06347.

Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. 2018. URL https://arxiv.org/abs/1506.02438.